

Smart City Traffic Intelligence

Part 2: Python Analytics Pipeline Report

1. Methodology- This project developed a reproducible Python-based traffic analytics workflow using the Metro Interstate Traffic Volume dataset. The workflow was first explored and tested interactively in Jupyter Notebook, then converted into reusable Python scripts for automation. The final solution consists of a main data pipeline, a feature-engineering module, a visualisation module and a command-line application. The pipeline processes the raw CSV from beginning to end, including schema validation, data cleaning, feature creation, figure generation and logging. Git and GitHub were used throughout the project to maintain incremental version history and support reproducibility.

2. Data Cleaning and Validation- The raw dataset contained 48,204 rows and nine original columns. The Python pipeline first validated that all expected columns were present before any transformation was performed. It then removed 17 exact duplicate rows, leaving 48,187 records. All date_time values were parsed successfully. The categorical fields were standardised, and missing holiday values were restored as the valid category "None". The pipeline also identified 10 impossible temperature readings of 0 Kelvin and replaced them using the median temperature for the corresponding month. In addition, one extreme rainfall value above 9,000 mm was identified and imputed using the monthly median rainfall. Numerical checks confirmed that snow values were non-negative, cloud coverage remained within the valid 0–100 range, and no negative traffic-volume values were present.

3. Feature Engineering- The cleaned dataset was transformed into a more machine-learning-ready format using NumPy and Pandas. Time-based features included Hour, Day of Week, and a Weekend Indicator. Because hour of day is cyclical, sine and cosine encodings were also created so that adjacent times such as 23:00 and 00:00 are represented as close values rather than distant numbers. Weather features included a Rain Indicator and one-hot encoded weather categories. Two continuous variables, temperature and cloud coverage, were scaled to a 0–1 range. A data-driven Congestion Category was also created using the 25th and 75th percentiles of traffic volume. Records at or below the 25th percentile were classified as Low, values above the 75th percentile were classified as High, and the remaining observations were classified as Medium.

4. Traffic Visualisation Findings- The cleaned dataset was transformed into a more machine-learning-ready format using NumPy and Pandas. Time-based features included Hour, Day of Week, and a Weekend Indicator. Because hour of day is cyclical, sine and cosine encodings were also created so that adjacent times such as 23:00 and 00:00 are represented as close values rather than distant numbers. Weather features included a Rain Indicator and one-hot encoded weather categories. Two continuous variables, temperature and cloud coverage, were scaled to a 0–1 range. A data-driven Congestion Category was also created using the 25th and 75th percentiles of traffic volume. Records at or below the 25th percentile were classified as Low, values above the 75th percentile were classified as High, and the remaining observations were classified as Medium.

5. CLI Application and Reproducibility- A command-line interface (CLI) application was developed to allow users to interact with the processed traffic dataset without modifying Python code. The application supports three commands: identifying the top five high-traffic hours, comparing weekday and weekend traffic, and querying traffic conditions for a specific date and time. The CLI also handles malformed date input gracefully by displaying a clear message and writing an ERROR entry to the log instead of producing an unhandled traceback. Logging is used throughout the project to provide an auditable processing trail. DEBUG messages record internal calculations, INFO messages record normal milestones and saved outputs, WARNING messages record recoverable data-quality changes, and ERROR messages record failures or invalid input. Git and GitHub were used to maintain incremental commits for each major task, making the workflow easier to reproduce and review.